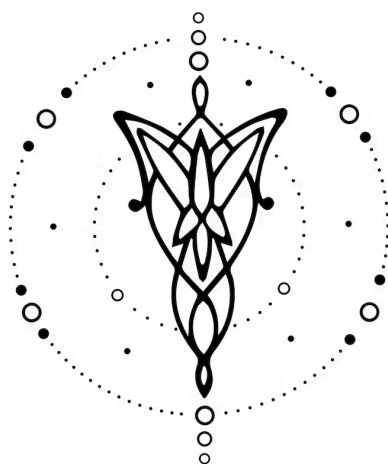




Bandailer-ER

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Diego	Mayansky	64.013	dmayansky@itba.edu.ar
Juan Pablo	Fernandez	64.650	juanfernandez@itba.edu.ar
Ian	Oniszczyk	64.984	ioniszczyk@itba.edu.ar
Mateo	Stella	65.780	mastella@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [Bandailer-ER](#).

3. Dominio

Desarrollar un lenguaje para crear modelos Entidad-Relación con componentes básicos como entidades, atributos tipados, claves primarias y únicas, y relaciones con sus cardinalidades y participaciones. Permitirá definir esquemas de manera textual, validar su consistencia (nombres únicos, referencias existentes, tipos compatibles y restricciones elementales de integridad) e inspeccionar el modelo durante su proceso de análisis y generación de diagramas.

Toda la ejecución se realizará dentro de la memoria del propio programa; los modelos y artefactos generados serán completamente virtuales y existirán únicamente dentro de la aplicación. El proceso se apoyará en un formalismo propio, sin noción real de motores de bases de datos, SQL, optimización física ni acceso al hardware subyacente, de modo que la evaluación del diseño sea lógica y determinística, reservando la interacción externa a funciones de registro y visualización.

La implementación satisfactoria de este lenguaje permitirá explorar alternativas de diseño conceptual, comparar ventajas y desventajas, y detectar inconsistencias o vulnerabilidades de modelado (por ejemplo, ausencia de claves, cardinalidades inválidas o referencias no resueltas) antes de obtener como resultado diagrama de Entidad-Relación.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Declaración de entidades con nombre único.
- (II). Se podrán declarar atributos con tipo (integer, decimal, string, bool, date, datetime, uuid, enum) y con modificadores como not null, unique, default, derived.
- (III). Declaración de claves primarias (una o compuesta).
- (IV). Declaración de relaciones n-arias con nombre, participantes (una o más entidades) y cardinalidades admitidas: 1:1, 1:N, N:1, N:M.
- (V). Declaración de relaciones con participación: total o parcial.
- (VI). Atributos de relación.
- (VII). Declaración de entidades débiles con relación identificante.
- (VIII). Comentarios de línea // ... y de bloque /* ... */.
- (IX). Realizar validaciones semánticas:
 - Referencia a entidades existentes.
 - No duplicación de nombres.
 - Existencia de primary key en cada entidad.
 - Cardinalidades válidas.
 - Tipos de atributos consistentes.
- (X). Exportación del modelo a un archivo .dot para generar el diagrama visual en Graphviz y también generar imágenes (SVG/PNG) directamente.
- (XI). Proveer operadores relacionales para restricciones y atributos derivados: <, >, =, !=, <=, >=.

- (XII). Proveer operaciones aritméticas básicas +, -, *, / y operadores lógicos AND, OR, NOT para restricciones y derivaciones.
- (XIII). Proveer expresiones condicionales (por ejemplo, `IF ... THEN ... OTHERWISE ...`) que permitan definir atributos derivados o restricciones cuyo valor dependa de condiciones lógicas, dentro de un contexto declarativo (no imperativo).
- (XIV). Proveer mecanismos para definir *assertions* sobre atributos y entidades, que permitan expresar condiciones lógicas o de integridad dentro del modelo.
- (XV). Proveer un mecanismo para definir herencia entre entidades.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que construya 1 esquema con 1 entidad que posea clave primaria y al menos un atributo adicional.
- (II). Un programa que construya 2 entidades y declare una relación 1:1 con participación total en un lado y opcional en el otro.
- (III). Un programa que construya 2 entidades y declare una relación N:M con atributos propios.
- (IV). Un programa que declare una relación ternaria entre tres entidades, con cardinalidades por rol y atributos propios, y que genere el diagrama correspondiente.
- (V). Un programa que declare una entidad con clave primaria compuesta formada por dos atributos existentes.
- (VI). Un programa que declare un atributo multivaluado en una entidad, lo valide semánticamente y lo represente correctamente en el diagrama según la notación seleccionada.
- (VII). Un programa que declare una entidad débil y su relación identificante con la entidad fuerte correspondiente.
- (VIII). Un programa que incluya una restricción de integridad tipo *assertion* válida sobre un atributo numérico.
- (IX). Un programa que defina una relación con participación total.
- (X). Un programa que tenga herencia de entidades.
- (XI). Un programa que contenga una entidad con un atributo derivado.
- (XII). Un programa que contenga una entidad con atributos *default*.
- (XIII). Un programa que haga uso de la estructura condicional IF/THEN/ELSE.
- (XIV). Un programa que haga uso del tipo de dato *enum*.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado que no cumple la sintaxis del lenguaje.
- (II). Un programa que declare dos entidades con el mismo nombre dentro del mismo esquema.
- (III). Un programa que declare una relación que referencia una entidad inexistente.

- (IV). Un programa que declare una entidad sin clave primaria.
- (V). Un programa que defina una cardinalidad inválida o mal formada en alguno de los participantes de una relación.
- (VI). Un programa que haga uso de variables o atributos no definidos.
- (VII). Un programa que intente realizar una operación aritmética con cadenas de texto.
- (VIII). Un programa que compare tipos de datos incompatibles.
- (IX). Un programa que declare más de una clave primaria en una entidad.
- (X). Un programa que declare una clave primaria con el modificador *primary* y otra clave primaria compuesta con la declaración PRIMARY.
- (XI). Un programa que tenga una entidad débil que no presente la palabra clave *weak*.
- (XII). Un programa que redefina la clave primaria por medio de la declaración PRIMARY.
- (XIII). Un programa que redefina la clave primaria por medio del modificador *primary*.
- (XIV). Un programa que contenga una entidad que se auto herede.
- (XV). Un programa que contenga herencia cíclica.
- (XVI). Un programa que redefina un atributo que se heredó de una entidad padre.
- (XVII). Un programa que tenga una relación unaria.
- (XVIII). Un programa que intente definir una clave primaria dentro de una relación.
- (XIX). Un programa que tenga una entidad débil pero que dicha entidad no posea una relación donde presenta participación total.
- (XX). Un programa que defina una clave primaria con un tipo de dato no soportado.
- (XXI). Un programa que defina un atributo default con un tipo de dato no soportado.

6. Ejemplos

En el **Cód. (1)**, se puede ver la creación de un modelo Entidad-Relación de RR.HH. con empleados, departamentos y asignaciones a proyectos.

```

schema Company // Todo el modelo vive dentro de este esquema.

// Departamento: identificador y nombre único.
entity Department {
  dept_id : uuid primary
  name    : string unique
}

entity Employee {
  eid      : uuid primary
  full_name : string not null
}

// Cada empleado pertenece a exactamente un departamento (participación TOTAL).
relationship belongs_to(Employee many, Department one total)

```

Código 1: Definición de entidades y una relación en un esquema de RR.HH.

En el **Cód. (2)**, se observa la creación de una entidad que modela un número de teléfono con clave primaria compuesta y haciendo uso de *assert* para validaciones.

```
schema Directory

entity Phone {
  area_code : integer
  number    : integer
  PRIMARY (area_code, number) // Clave primaria compuesta
  owner     : string not null  // Dueño de la línea.

  // Reglas sencillas para validar rangos (opcionales):
  assert area_code >= 100 AND area_code <= 999
  assert number > 0
}
```

Código 2: Declaración de una entidad con clave primaria compuesta y restricciones de validación.

En el **Cód. (3)**, se ejemplifica una relación con atributos propios y una expresión condicional derivada, que ajusta el valor de un atributo en función de otro dentro de la misma relación.

```
schema Projects

entity Employee {
  eid      : uuid   primary
  name     : string not null
}

entity Project {
  pid      : uuid   primary
  title    : string not null
  deadline : date
}

// Relación con atributo "hours" y derivación condicional.
relationship assigned_to(Employee many, Project) {
  hours : decimal not null
  role  : string derived =
    IF hours > 40 THEN "Lead"
    OTHERWISE "Contributor"
}
```

Código 3: Declaración de una entidad con clave primaria compuesta y restricciones de validación.

En el **Cód. (4)**, se muestra la definición de tres entidades que utilizan herencia para definir una relación de parentesco.

```
schema Company

entity Persona {
  id      : uuid primary
  name    : string not null
}

entity Empleado : Persona { // Empleado hereda de Persona
  salary : decimal not null
}

entity Gerente : Empleado { // Gerente hereda de Empleado
  bonus  : decimal
}
```

Código 4: Declaración de entidades hijas por medio de herencia

En el **Cód. (5)**, se muestra la definición de una relación ternaria entre entidades distintas, con diferentes cardinalidades y tipos de participación, siendo una de ellas una entidad débil.

```
schema University

entity Student {
  sid : uuid primary
  name : string not null
}

entity Course {
  cid : uuid primary
  title : string not null
}

// La inscripción depende de Student y Course, por lo tanto es una entidad débil.
entity Enrollment {
  semester : string not null
  PRIMARY (sid, Course:cid, semester)
} // Notar que el prefijo "Course:" se puede obviar porque no hay conflictos

// Relación ternaria: un profesor dicta un curso en el cual los estudiantes se inscriben.
relationship teaches(Student one, Course many, Enrollment many total) {
  classroom : string not null
  schedule : string
}
```

Código 5: Declaración de relación ternaria